ÜZEMELTETÉSI KÉZIKÖNYV

OpenHDID – Ügyfélszolgálati hívóazonosítás

A konténeres OpenHDID napi üzemeltetése: ellenőrzés, szolgáltatások kezelése, naplók és felügyelet, ütemezett feladatok, a leggyakoribb helyzetek, mentés és visszaállítás, frissítés, skálázás és biztonsági karbantartás.

VERZIÓ
1.14.0 · 2026. október 26.

KIADÁS
OpenHDID 1.14.0 · ghcr.io/adamfhun/openhdid

CÉLKÖZÖNSÉG
rendszerüzemeltető, rendszergazda

KAPCSOLÓDÓ
telepítési útmutató, felhasználói kézikönyv

TARTALOM

1 Napi és heti ellenőrzés
2 Szolgáltatások kezelése
3 Naplók és felügyelet
4 Ütemezett feladatok
5 Beállítások és titkok módosítása
6 Üzemeltetési helyzetek
7 Mentés és visszaállítás
8 Frissítés és visszaállás
9 Skálázás és teljesítmény
10 Biztonsági karbantartás
11 Adatmegőrzés
12 Parancsreferencia
R Rövidítésjegyzék

Minden parancs a telepítési mappából (például `/opt/openhdid`) futtatandó, ahol a `compose.yaml`, a `.env` és a `db.env` található. Az alkalmazás parancsainak előtagja `docker compose exec web php artisan`.

1 Napi és heti ellenőrzés

A legtöbb kérdésre az *Adminisztráció › Rendszer › Rendszerállapot* oldal válaszol. Minden összetevő állapota színes jelvénnyel látszik, lenyitható tanáccsal; a fejléc a futó verziót mutatja. Ugyanezek az ellenőrzések gépi úton:

- `GET /health`: 200, ha semmi nem hibás, 503, ha bármi hibás; csak a felügyeleti címekről érhető el, lásd `OPENHDID_MONITORING_ALLOW`;
- `docker compose exec web php artisan hdid:health`: kilépési kód 0 rendben, 1 figyelmeztetés (csak `--strict` mellett), 2 hiba; `--json` géppel olvasható.

Naponta:

- `docker compose ps`: minden szolgáltatás fut, a `web` `healthy`, a `migrate` `exited (0)`;
- a Rendszerállapot kártyái zöldek:
  - a lap két blokkra oszlik: fent „A rendszer működése”, lent „Üzemmódok és beállítások”; az alsó blokk kék „Tájékoztatás” jelvénye (élő csomag-felülírás, több ügyfélnél rögzített „megosztott” telefonszám, bekapcsolt csak prémium mód) a normális működés része, teendő nincs; sárga ott az, amit élesítés előtt ki kell kapcsolni (belépési hivatkozás előnézete, kikapcsolt ügyféltörzs-kötelezettség, ADFS-hibakeresés); a nem zöld kártyán hivatkozás vezet a kezelő oldalra;
  - sárga (felső blokk): a „Kimenő üzenetek” kártya nem kézbesíthető SMS vagy nem futott kézbesítési lekérdezés mellett, a „Belépési kötések” kártya, ha az elmúlt 7 napban egy céges belépés a fiók címére más címtár-azonosítóval érkezett (6. fejezet), naplózó levelező, a „Feladatsor” kártya, ha a `sync-worker` nem ad életjelet vagy sikertelen feladat van (az *Üzenetsorok* oldal kezeli), hiányzó API-kulcs, a panelről indított, 5 percnél régebben várakozó átvétel, az „Ügyféltörzs-átvétel” kártyán a következő átvételnél lezáruló fiókok (a tételük az utolsó futásból hiányzott: az Ügyféltörzs (EMD) lista „A következő átvételnél lezárul” fülén ellenőrizhető, mielőtt a következő futás lezárja őket), vagy 120 és 300 mp közötti PHP időkorlát;
  - sárga a *Kapacitás* kártya, ha az aktív fiókok mai csúcsa eléri a használati profil tervezett értékének 80 %-át vagy az adatbázis-kapcsolatok 80 %-a használatban: a következő profilra kell lépni (9. fejezet); sárga az *Auditnapló* kártya, ha a beállított kereten túli bejegyzések még az adatbázisban vannak, mert az éjjeli archiválás nem futott (11. fejezet);
  - a 2026. októberi kártyák: a *Valkey / Redis* kártya piros kiszorító politikánál (a fő Valkey-n a `maxmemory-policy noeviction` kötelező) vagy elérhetetlen tárnál, sárga 80 % fölött, `maxmemory` nélkül, kiszorított kulcsnál és külső Valkey-n az alapértelmezett `REDIS_PREFIX` mellett, tájékoztat, ha a szolgáltató az adatokat nem adja ki; a *Munkamenet-tár* sárga, ha a munkamenetek a fő Valkey-ben vannak (üres `REDIS_SESSION_HOST`), ha a politika nem `allkeys-lru`, vagy ha az elmúlt órában munkamenetek szorultak ki (áradat vagy kicsi tár), piros, ha nem érhető el; a *Demóadatok* piros éles környezetben demóadattal; az alsó blokk *Korlátok alól mentesített hálózatok* kártyája a `HDID_RATE_LIMIT_EXEMPT_NETWORKS` tartományait sorolja (hibás listánál piros). A *Kimenő üzenetek* kártya sárga, ha egy üzenet 5 percnél, egy belépési kód 2 percnél régebben vár, ha egy belépési plafon betelt (a csatornával és az ablakkal), és ha az elsőbbségi sornak nincs feldolgozója; a *Levelezés* piros, ha az elmúlt 24 órában SMTP-küldés STARTTLS hiányában megállt, sárga `MAIL_REQUIRE_TLS=false` mellett; az *Egyszeri bejelentkezés* sárga, ha egy ügyfél-Entra-bérlő domainje munkatársi domain, vagy egyik ügyféltörzs-domainlistán sincs (6. fejezet). Az ütemező és a sor kártyája piros, ha egy másik telepítés ugyanazt a Valkey-t és kulcselőtagot használja (6. fejezet);
  - piros: nem futó feldolgozó vagy ütemező, adatbázis-hiba, élesben naplózó SMS, a várt időköz kétszeresénél régebbi utolsó sikeres átvétel, vagy 120 mp alatti PHP időkorlát (a képbe épített `php.ini` 300 mp-et ad; ennél rövidebb felülírás mellett az átvétel el sem indul, és a konténer indításkori ellenőrzése is megáll);

- *Üzenetsorok* (Rendszer): a `priority`, a `default` és a `sync` sor feldolgozója friss életjelet ad (közös Valkey-n az oldal figyelmeztet), nincs sikertelen feladat, a várakozók száma nem nő; alatta a *Kimenő üzenetek* (ugyanez a lista a Tartalom alatt is): nincs „sikertelen” vagy órák óta „várakozik” tétel;
- *Ügyféltörzs-átvételek*: az utolsó átvétel sikeres, a lezárt tételek száma nem ugrott meg.

Hetente:

- az auditnapló `login.rejected` (az `sso_subject_mismatch` és `sso_subject_taken` ok a címtár-azonosító változását jelzi, lásd a 6. fejezetet), `sso.subject_unbound`, `login.request_throttled`, `account.locked`, `api_key.rejected` (hálózati címenként és okonként percenként egy sor), `message.failed` és `message.refused` (kézbesítetlen levél vagy SMS, illetve az SMS-keret miatt elutasított belépőkód-kérés) és `id_session.manual` eseményei; a 2026. októberi kiadástól a szerepkör- és jogváltozások (`role.updated`, `role.permission_change_rejected`, `user.roles_changed`, `user.role_escalation_rejected`, `user.manage_rejected`), a belépési plafonok (`login.request_capped`), az SMS célszám miatti elutasítások (`message.refused` `sms_destination`, `sms_limit_per_day`), a kérdés-válasz zárolások (`id_session.qa_refused`, `client.qa_unlocked`), a kézi e-mail-cserék (`account.access_revoked` `email_changed_by_admin`, `client.holder_data_removed`) és a visszaállítás utáni kiléptetés (`access.revoked_all`);
- a lemezterület: `docker system df`, és a kötetek mérete;
- a heti sérülékenységvizsgálat eredménye a kiadott image-en (10. fejezet).

2 Szolgáltatások kezelése

TEENDŐ	PARANCS
Állapot	`docker compose ps`
Indítás, módosítás érvényesítése	`docker compose up -d` (csak a változott konténereket indítja újra)
Egy szolgáltatás újraindítása	`docker compose restart web` (vagy `worker`, `sync-worker`, `scheduler`)
Leállítás (adatok megmaradnak)	`docker compose stop`, illetve `docker compose down` (a kötetek maradnak; a `-v` kapcsolót soha ne használja élesben)
Alkalmazásparancs	`docker compose exec web php artisan <parancs>`
Parancs új, ideiglenes konténerben	`docker compose run --rm web artisan <parancs>`
nginx újratöltése (tanúsítványcsere)	`docker compose exec web openhdid reload`
Tanúsítvány ellenőrzése	`docker compose exec web openhdid tls-check`
Verzió	`docker compose exec web openhdid version`
Karbantartási mód	`docker compose exec web php artisan down --retry=30`, majd `… up`

Újraindítás. A konténerek újraindítási szabálya `unless-stopped`: leállás vagy a gazdagép újraindítása után maguktól elindulnak. A `web` két folyamata (nginx és PHP-FPM) egymást felügyeli: ha az egyik leáll, a konténer is kilép, és a Docker újraindítja. Leállításkor mindkettő befejezi a futó kéréseket.

Karbantartási mód. `APP_MAINTENANCE_DRIVER=cache` mellett minden webpéldányra hat. Alatta a portál, a panel és a telefonközponti végpontok is „karbantartás” választ adnak; időzítse forgalmon kívülre.

3 Naplók és felügyelet

FORRÁS	HOL	MIRE JÓ
Webkiszolgáló	`docker compose logs web`: JSON access log (idő, IP, kérésazonosító, metódus, státusz, méret, időtartam), nginx- és PHP-FPM-hibák	forgalom, hibaarány, lassú kérések; URL, lekérdezés és süti nincs benne
Alkalmazás	`docker compose logs web worker sync-worker scheduler`	kivételek, figyelmeztetések (`LOG_LEVEL=warning`), naplózó módban a kimenő üzenetek
Indítás és beállítás	a konténerek első sorai: `openhdid:` előtaggal	beállítási hibák, tanúsítvány-összefoglaló, figyelmeztetések
Auditnapló	felület: *Rendszer › Auditnapló* (szűrők, CSV-export)	ki, mikor, mit: belépések, elutasítások, beállítások, azonosítások, kulcsok
Rendszerállapot	felület, `/health`, `hdid:health`	összetevők, életjelek, sorhossz, átvétel kora, megőrzés

```
docker compose logs -f --tail=200 web
docker compose logs --since 30m --no-log-prefix web | grep '"status":5'
docker compose logs --since 1d --no-log-prefix web | grep -v '^{'      # hibák, figyelmeztetések
docker compose logs --no-log-prefix web | grep '"status":429' | wc -l  # forgalomkorlát-elutasítások
docker compose logs --no-log-prefix web | grep 'limiting requests'     # az nginx kéréskorlátja (cím szerint)
```

Forgatás és gyűjtés.

- **Forgatás:** a naplókat a konténermotor forgatja (`json-file` meghajtó, 10 × 20 MB szolgáltatásonként; Podman alatt `k8s-file`, csak méretkorláttal, a `compose.yaml` `x-logging` blokkjában).
- **Központi gyűjtés:** ugyanitt állítható a `journald`, `syslog`, `gelf` vagy `fluentd` meghajtó. A napló titkosított csatornán, a futtatási fióktól elkülönített, módosítás ellen védett tárba kerüljön.
- **Hibasorok:** az nginx és az alkalmazás hibasoraiban URL vagy személyes adat lehet: korlátozott hozzáférés, rövid nyers megőrzés.

Külső felügyelet. Ajánlott figyelni:

- a `/health` választ (503 = riasztás);

- a konténerek egészségét (`docker compose ps` , `health_status` események);
- a lemezterületet és a tanúsítvány lejáratát (a web naplója 30 nappal előtte figyelmeztet);
- a kiugró 4xx/5xx arányt.

Egyszerű gazdagépes riasztás, ha nincs felügyeleti rendszer:

```
*/5 * * * * cd /opt/openhdid && docker compose exec -T web php artisan hdid:health
>/dev/null || echo "OpenHDID: hibás rendszerállapot" | mail -s "OpenHDID"
uzemeltetes@example.org
```

---

4 Ütemezett feladatok

A cron szerepét a `scheduler` szolgáltatás tölti be. A gazdagépen nincs crontab-bejegyzés, és a konténerekben sincs cron-démon. A feladatok listája és a következő futásuk:

```
docker compose exec scheduler php artisan schedule:list
```

- **Percenként:** lejárt azonosítási munkamenetek zárása; életjelek; elakadt üzenetek újrasorolása; az esedékes SMS kézbesítési lekérdezések.
- **15 percenként:** ügyféltörzs-token frissítése, ha be van kapcsolva.
- **Óránként:** csomag-felülírások lejárata; befejezett hívások tisztítása; beragadt hívások lezárása.
- **Naponta 03:30-kor:** adatmegőrzési tisztítás és az auditnapló archiválása (`hdid:prune` , háttérben, hogy ne tartsa fel az ütemezőt; futásonként legfeljebb 120 napot archivál).
- **Az `EMD_SYNC_CRON` szerint:** teljes ügyféltörzs-átvétel.

Időzóna: `APP_TIMEZONE` .

Az ütemező vagy a feldolgozó életjele hiányzik (piros „Ütemező” vagy „Feladatsor” kártya):

1. `docker compose ps scheduler worker` : fut-e; ha újraindulgat, `docker compose logs --tail=100 scheduler` . Jellemzően adatbázis-kapcsolat vagy beállítási hiba az ok, a konténer első soraiban.
2. A kártyák a következő percben zöldre váltanak, amint az életjel megérkezik.
3. Az ütemezőből pontosan egy példány fusson; skálázáskor se növelje.

A panelről indított átvétel „várakozik” állapotban marad: a `sync-worker` nem fut, vagy az `EMD_SYNC_QUEUE` nem egyezik a figyelt sorral (alapból mindkettő `sync`). A Rendszerállapot 5 perc után figyelmeztet. A panelről feltöltött fájl is a `sync-worker` -ben dolgozódik fel (1.2.0 óta), ugyanezzel a feltétellel.

Egy átvétel „Sikertelen”, a hibaszöveg szerint a PHP félbeszakította: a futást az időkorlát, a memória vagy egy összeomlás vágta el; a már beírt tételek megmaradtak, lezárás nem történt, a zár felszabadult. Nézze meg a `sync-worker` naplóját (`docker compose logs --tail=200 sync-worker`) és a Rendszerállapot „PHP időkorlát” kártyáját; a feldolgozó 1200 másodperces job-időkorlátja nagy állománynál is elegendő, ennél hosszabb futás a forrást vagy a memóriát jelzi.

5 Beállítások és titkok módosítása

- **Telepítési érték** (`.env`): szerkesztés, majd `docker compose up -d` . Az érintett konténerek újraindulnak, a konfigurációs gyorsítótár újraépül; ha egy érték hibás, a konténer nem indul el, az oka a naplóban.
- **Üzleti paraméter** (Beállítások oldal): azonnal él minden folyamatban, újraindítás nélkül.
- **Titokcsere** (adatbázis, EWS, Ozeki, SSO, ügyféltörzs): előbb a partner oldalán, majd a `.env` -ben (vagy a titokfájlban, `*_FILE`), végül `docker compose up -d` .
- **Az adatbázis-jelszó cseréje** a beépített MariaDB-ben:

  ```
  docker compose exec mariadb sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb -uroot -e
  "ALTER USER \`openhdid\`@\`%\` IDENTIFIED BY \"új-jelszó\";"'
  ```

  Utána a `.env` `DB_PASSWORD` értéke és `docker compose up -d` .
- **Az ügyféltörzs tokenjei** jelszó- vagy végpontcsere után: `docker compose exec web php artisan hdid:emd-sync-token` .
- **Hibakeresés részletes hibaüzenetekkel.** Éles környezetben (`APP_ENV=production`) az `APP_DEBUG=true` tiltott: a konténer 64-es kilépési kóddal megáll, mert a hibaoldal belső részleteket mutatna. Egy hibakeresési időszakra a `.env` -ben `APP_ENV=staging` és `APP_DEBUG=true` együtt, majd `docker compose up -d` ; a végén mindkettő visszaállítása és újra `docker compose up -d` . Ha csak több napló kell, az `APP_ENV` maradhat, elég a `LOG_LEVEL=debug` .

> **APP_KEY**
> Meglévő telepítésen nem cserélhető a szokásos módon. A titkosított válaszok, kódmásolatok, API-tokenek, sütik és kulcsolt lenyomatok mind tőle függnek; cseréje külön adatmigrációs tervet és visszaállítási próbát igényel.

6 Üzemeltetési helyzetek

Az alkalmazás működésével kapcsolatos helyzetek (belépés, azonosítás, hívások, jogosultságok) a felhasználói kézikönyvben és a Rendszerállapot kártyáinak tanácsaiban szerepelnek. Ez a fejezet a konténeres üzemre jellemző eseteket írja le.

Az összes diagnosztikai parancs egy helyen, másolható formában: a kiadás `TROUBLESHOOTING.md` mellékletében (állapot, újrainduló szolgáltatás, proxy, Docker-jogosultság, fájltulajdonos, ügyféltörzs-elérés és -belépés, tanúsítvány-lánc és belső CA, egyszeri bejelentkezés, a belépés elutasításának oka, levél és SMS, mentés).

A konténer nem indul el vagy újraindulgat

1. `docker compose ps` és `docker compose logs --tail=50 <szolgáltatás>` . Az indítószkript `openhdid: ERROR:` sora megnevezi az okot: hiányzó vagy hibás `APP_KEY` , nem https `APP_URL` , hiányzó adatbázis-adat, élesben tiltott beállítás, hibás kimenő proxycím (`HDID_HTTP_PROXY` ; az értéket a napló nem írja ki), érvénytelen vagy túl tág `HDID_RATE_LIMIT_EXEMPT_NETWORKS` -elem, hibás `OPENHDID_NGINX_*` vagy `OPENHDID_FPM_*` érték, tanúsítványhiba, nem írható `storage` kötet.

2. Ha csak a `web` indul újra („restarting”), az ok a tanúsítvány vagy az nginx: a `docker compose run --rm --no-deps web tls-check` futó `web` nélkül is ellenőrzi a fájlokat (hiányzó `fullchain.pem` / `privkey.pem`, nem olvasható kulcs, névegyezés, csonka lánc). Az `artisan` parancsok ekkor is futnak: `docker compose run --rm --no-deps web artisan hdid:make-admin …`.
3. A `migrate` hibájánál a többi szolgáltatás nem indul: `docker compose logs migrate`. Jellemző okok: az adatbázis nem érhető el 120 másodpercen belül, hibás migrációs jelszó a `db.env`-ben, a migrációs fióknak nincs joga, vagy éles környezetben demóadat van az adatbázisban (lásd „Demóadat éles környezetben”).
4. Javítás után: `docker compose up -d`.

Az image letöltése időtúllépéssel elhal

Ha a `docker compose pull` 30 másodperc után időtúllépést ad, miközben a `curl` a gazdagépről eléri a `ghcr.io`-t, a Docker-démon nem látja a felhasználó proxybeállítását (`env | grep -i proxy`). A démon a proxyt systemd drop-inból kapja:

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf >/dev/null <<EOF
[Service]
Environment="HTTP_PROXY=${http_proxy:-$HTTP_PROXY}"
"HTTPS_PROXY=${https_proxy:-$HTTPS_PROXY}"
"NO_PROXY=localhost,127.0.0.1,${no_proxy:-$NO_PROXY}"
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
docker info | grep -i proxy
```

Ha nincs proxy, az IPv6 lehet félig beállítva: a `curl -4` és a `curl -6` a `https://ghcr.io/v2/` címre mutatja meg (`HTTP/2 401` a jó válasz); az IPv4 előnyben részesítése a `/etc/gai.conf` `precedence ::ffff:0:0/96 100` sorával.

Az alkalmazás kimenő kérései proxy mögött

Az image letöltése a Docker-démon proxyján megy (előző pont), az alkalmazás kimenő kérései (ügyféltörzs, Exchange, Ozeki SMS, ADFS, Entra) viszont a `.env` `HDID_HTTP_PROXY` kulcsán; a konténer `HTTP_PROXY` / `HTTPS_PROXY` változóit az alkalmazás nem használja. A használt proxyt (jelszó nélkül) és a kivételeket a Rendszerállapot „Kimenő proxy” kártyája mutatja, parancssorból:

```
docker compose exec web php artisan tinker --execute '$c =
app(App\System\HealthChecks::class)->outboundProxy(); echo $c->status->value, ": ", $c-
>detail, PHP_EOL;'
```

Az ügyféltörzs kapcsolódási hibája proxyn át a proxyt is megnevezi („…, proxy: http://proxy.example.org:3128: …”), és az okot:

- **a proxy elutasította a kapcsolatot vagy nem érhető el, illetve a neve nem oldható fel:** a `HDID_HTTP_PROXY` gépneve vagy portja hibás, vagy a tűzfal nem engedi a konténerből;
- **a proxy felhasználónevet és jelszót kér (407):** `http://felhasznalo:jelszo@gep:port` alakban kell megadni, a különleges karakterek %-kódolva (`@` = `%40` , `$` = `%24` , `#` = `%23`); Windows-hitelesítésű (NTLM) proxy nem támogatott;

- **a proxy nem engedi a címet (403):** a proxy gazdájának kell engedélyeznie, belső kiszolgálónál a `HDID_NO_PROXY` a megoldás;
- **a proxy nem érte el a címet (5xx):** jellemzően belső kiszolgáló, amelyet a proxy nem lát: vegye fel a `HDID_NO_PROXY` listába (név, `.domain`, IP-cím vagy `10.0.0.0/8` alakú tartomány).

A `.env` módosítása után `docker compose up -d`; a konténerek az új értékkel indulnak újra. Az ügyféltörzs-címek proxytudatos ellenőrzése a `TROUBLESHOOTING.md` 8. pontjában áll.

„permission denied ... docker.sock”

A futtató felhasználó nincs a `docker` csoportban, vagy a tagság még nem él: `sudo usermod -aG docker <felhasználó>`, majd új bejelentkezés (vagy `newgrp docker`); ellenőrzés `id` és `docker ps`. Ha a Docker kliens a Podman socketjét keresi, a korábbi `DOCKER_HOST` változót kell törölni (`unset DOCKER_HOST`, a profilból is), és a `docker context use default` parancsot futtatni. Rootless Dockernél nincs `docker` csoport: a démon a felhasználó saját egysége (`systemctl --user status docker`), a kliensnek a `DOCKER_HOST=unix:///run/user/<uid>/docker.sock` változó kell; ha hiányzik, a kliens a root démon socketjét keresi, és ugyanezt a hibát adja (telepítési útmutató 3. fejezet, „Rootless Docker”).

Belépés elutasítva „hibás felhasználónév vagy jelszó” üzenettel

A felület minden elutasításra ugyanezt a semleges üzenetet adja. A valódi ok az auditnaplóban van (`login.rejected` esemény); a panel nélkül a konténerből kérdezhető le:

```
docker compose exec web php artisan tinker --execute 'App\Models\AuditLog::query()->where("event", "login.rejected")->latest()->take(5)->get()->each(fn ($e) => print($e->created_at." ".json_encode($e->context).PHP_EOL));'
```

A `reason` mező értékei: `invalid_credentials` (rossz e-mail vagy jelszó; a `hdid:make-admin` újrafuttatása ugyanarra az e-mailre a jelszót felülírja, bekapcsolt kétlépcsős belépésnél `--force`-szal), `account_locked` (fokozódó zárolás, várni kell), `no_permission` (nincs panel-jogosultság), `method_disabled` (a jelszavas belépés kikapcsolva a beállításokban), `no_external_record` és `external_record_mismatch` (hiányzó vagy eltérő ügyféltörzs-tétel), `account_closed`, `demo_account` (demófiók éles környezetben). Ha egy sor sem keletkezett, a kérés el sem jutott a belépési szabályig: IP-korlát (percenként 5 próba; vállalati NAT mögött lásd „Vállalati hálózat a korlátokba ütközik”) vagy lejárt munkamenet (419). A `invalid_token` ok „Token is longer than 16384 bytes.” részlettel azt jelenti, hogy a szolgáltató túl hosszú tokent küldött (jellemzően túl sok csoport-igénnyel): a kiadási szabályokat kell szűkíteni.

Ügyfél SMS-es belépése zárolva, pedig nem ő próbálkozott. A belépőkódot bárki kérheti, aki az ügyfél e-mail-címét ismeri, így egy idegen egy kódkéréssel és öt hibás tippel az ügyfél SMS-es belépését 120 percre lezárhatja. Ez dokumentált kivétel (megrendelői döntés): a zárolás csak az SMS-utat érinti, a belépési hivatkozás és a céges bejelentkezés közben működik, és minden ilyen zároláshoz egy kimenő kód kell, amelyet a számonkénti (percenként 1, óránként 3, naponta 5) és a rendszerszintű kódkeret korlátoz. Teendő: az ügyfél lépjen be hivatkozással, a zárolást az adatlapon a `Zárolás feloldása` oldja fel, a `login.rejected` sorok forrás-IP-je a webszerver naplójával vethető össze.

Kétlépcsős belépés. Ha be van kapcsolva, a jelszó vagy a céges bejelentkezés után a *Belépés megerősítése* oldal kéri a második faktort; a hibák az `mfa.failed`, a zárolás az `account.mfa_locked` eseményben látszik. Elveszett faktornál a Felhasználók listán `Kétlépcsős belépés nullázása`, vagy `docker compose exec web php artisan hdid:mfa-reset email@example.org`; ha senki nem

tud belépni (például csak SMS engedélyezett, és az átjáró nem működik), a `docker compose exec web php artisan hdid:mfa-reset --off` kikapcsolja. Az ügyféltörzs HTTP 4xx/5xx válaszának törzse a `docker compose logs worker` kimenetében „EMD export refused” sorként olvasható.

Az ADFS-belépés nem találja a fiókot

Az ADFS-belépés oka a felületen is megjelenik; a „nem küldött használható e-mail-címet” üzenet a `no_email_claim` ok: a `USER_ADFS_EMAIL_CLAIM` / `CLIENT_ADFS_EMAIL_CLAIM` igényeinek (claimjeinek) egyike sem adott e-mail-címet (üresen `emailaddress`, `email`, `upn`, `preferred_username`, `unique_name`). A fenti paranccsal kiolvasott bejegyzés értékek nélkül mutatja:

- `claims_checked` és `claims_passed_over`: a keresett igények és mindegyik oka (`missing`, `empty`, `domain_account`, `not_an_email`, `several_addresses`);
- `id_token_claims`, `access_token_claims`: a tokenekben ténylegesen érkezett igények neve;
- `access_token`: `read`, ha az access token igényei is számítottak; `ignored: …` az okkal (például `issued to another client`), `not_a_jwt` vagy `absent`;
- sikeres vagy `no_account` belépésnél `email_claim` és `email_token`: melyik igény és melyik token adta az összevetett címet;
- ha az ADFS maga utasította vissza a kérést: `provider_error` és `provider_error_description` (MSIS-kóddal), például a `USER_ADFS_SCOPES` nem engedélyezett hatóköre miatt, vagy mert a `USER_ADFS_RESOURCE` nem egyezik a Web API azonosítójával, illetve a kliens nincs engedélyezve a Web API-n.

Az e-mail-igény egyik tokenben sincs, a döntő igény a `upn` (`USER_ADFS_EMAIL_CLAIM=email` mellett `no_email_claim`): ha az `access_token_claims` csak `apptype`, `appid`, `authmethod`, `ver`, `scp`, `sub` és az időbélyegek, az access token az ADFS saját userinfo-végpontjának szól (közönsége `urn:microsoft:userinfo`, a hibakeresési naplóban látszik), és ilyenkor a Web API kiadási szabályai nem futnak. Teendő: Web API az ADFS-alkalmazáscsoportban az e-mail-szabállyal és a kliens engedélyével, az azonosítója a `USER_ADFS_RESOURCE` (telepítési útmutató, „ADFS beállítása lépésenként”). Ha a közönség már a Web API, de az `email` igény `missing`, a kiadási szabály hiányzik; ha `empty`, a felhasználó levelezési címe (`mail` attribútum) üres az AD-ben.

Több Entra-bérlő (`CLIENT_ENTRA_TENANTS`): a `login.rejected` bejegyzés `ID token tenant not allowed` hibája a listán kívüli bérlőt jelent (vendégfelhasználónál a saját bérlője számít); a `tenant_domain_not_allowed` ok a bérlő domain-szűrőjén elutasított címet (`tenant`, `allowed_domains` mezők); másik engedélyezett bérlőből érkező azonos cím `sso_subject_mismatch` `other_tenant` részlettel. Az „Egyszeri bejelentkezés” (oidc) kártya a bérlők számát és domainjeit mutatja, sárga, amíg a `CLIENT_ENTRA_TENANT` és a `CLIENT_ENTRA_TENANTS` is ki van töltve, és akkor is, ha egy bérlő domainje munkatársi domain, vagy egyik ügyféltörzs-domainlistán sincs: ilyenkor az a domain vagy nem lép be ügyfélként, vagy a szinkron kihagyja a tételeit. Teendő: a domain felvétele az *Ügyféltörzs (EMD) › Besorolás › Ügyfél e-mail-domainek* közé (és levétele a munkatársiak közül), vagy törlése a bérlő mellől; a sárga kártya a `hdid:health --strict` kilépési kódját is rontja. A bérlők és domainjeik állapota a *Beállítások › Ügyfélportál › Belépési módok* blokkban, a „Céges belépés: Microsoft Entra ID” kapcsoló alatt látszik, domainenként színes jelvénnyel (zöld: ügyfél-domain a tételszámmal; sárga: még nincs tétel, vagy listán kívüli domain `CLIENT_LOGIN_REQUIRE_EMD=false` mellett; piros: munkatársi domain, vagy listán kívüli, ha tétel kell).

A beállítás idejére a `.env` `HDID_ADFS_DEBUG=true` értéke (utána `docker compose up -d`) minden ADFS-belépésnél a naplóba írja (`docker compose logs web | grep 'ADFS debug'`) az engedélykérést, a tokenválaszt a két token fejlécével és tartalmával az érkezett alakban (`received`) és visszafejtve (`header`, `claims`), valamint a döntést; az aláírásnak csak a hossza látszik, a frissítő token nem kerül bele, így a naplóban nincs újra felhasználható token. A tokenek személyes adatot tartalmaznak: utána állítsa vissza `false`-ra. Amíg be van kapcsolva, a rendszerállapot „Belépési szolgáltatók” csempéje sárga.

A céges belépés „más címtár-azonosító” okkal elutasul

A céges belépés az első alkalomtól a szolgáltató állandó azonosítójához kötött (Entra `oid`, ADFS `sub`); a fiók e-mail-címére (vagy dolgozói azonosítójára) más azonosítóval érkező belépés `sso_subject_mismatch`, a már más fiókhoz kötött azonosító `sso_subject_taken` okkal elutasul, a felületen a kimondott üzenettel; a fiók nem zárolódik. Tipikus okok: a kilépett kolléga címét új belépő kapta, a címtárban átírták a levelezési címet vagy a UPN-t, a címtárfiókot törölték és újra létrehozták. Az auditbejegyzés a szolgáltatót és az igény nevét adja, az azonosító értékét nem; a `bound_principal_id` a már kötött fiók. Lezárt vagy törölt fiók kötése nem akadály: azt a rendszer magától átadja az új fióknak (`sso.subject_unbound`, `holder_closed` vagy `holder_deleted` okkal); az újranyitott régi fiók kapja utána az elutasítást.

Teendő: ha ugyanaz a személy, a Felhasználók vagy Ügyfelek lista sormenüjében, illetve a szerkesztő oldal vagy adatlap fejlécében `Belépési kötés törlése` (munkatársnál a munkatársak kezelésének jogával, főrendszergazdáét csak főrendszergazda; ügyfélnél az ügyfelek kezelésének jogával), vagy `docker compose exec web php artisan hdid:sso-unbind email@example.org`; a következő belépés újra köt (`sso.subject_bound`). Ha nem ugyanaz a személy, a kötés marad, és az ügyféltörzsben kell rendezni, kié a cím. ADFS újraregisztrálása (új kliensazonosító) után a kötést a következő belépés magától megújítja; minden kötés törlése: `docker compose exec web php artisan hdid:sso-unbind --all --provider=adfs --force`. A kötést a szinkron, a lezárás, a visszavonás és az MFA-nullázás nem érinti. A Rendszerállapot „Belépési kötések” kártyája és az adatlap „Belépési kötés” mezője mutatja az elutasításokat. Az `oid` / `sub` nélkül érkező token `no_subject_claim` okkal elutasul (`HDID_SSO_REQUIRE_SUBJECT=false` engedi kötés nélkül); dolgozói azonosító módban a hiányzó igény `no_employee_id_claim`, a bejegyzés `employee_id_claim` és `employee_id_problem` mezővel. Kilépéskor az alkalmazás csak a saját munkamenetét és tokenjét szünteti meg, a Microsoft- vagy ADFS-munkamenetet nem: közös gépen a munkaállomás zárolása a védelem.

Tanúsítványhiba vagy lejárat

A `web` indításkor ellenőrzi a tanúsítványt. A `docker compose exec web openhdid tls-check` kiírja:

- az alanyt, a kiállítót és a lejáratot;
- a kulcs típusát és a láncot;
- a lefedett neveket és a hibát.

Csere: új fájlok a `tls` mappába (tulajdonos a konténer 82-es felhasználója, a kulcs 400-as joggal; rootless Dockernél a tulajdonost a konténer névterén belül kell beállítani, mert a 82 a gazdagépen az altartomány `kezdő + 81` azonosítója, például `165617`, lásd a telepítési útmutató 3. fejezetének „Rootless Docker” részét), majd `docker compose exec web openhdid reload`. Ha a web nem fut, a javítás után `docker compose up -d`.

Nem mennek ki az e-mailek vagy az SMS-ek

1. *Rendszer › Üzenetsorok* (vagy *Tartalom › Kimenő üzenetek*): „várakozik” állapotnál a `worker` nem fut (az oldal soronként mutatja az utolsó életjelet; `docker compose ps worker`, `docker compose logs --since 1h worker`); amíg áll, a `Várakozók küldése most` gomb a böngészőből, feldolgozó nélkül küldi el a legrégebben várakozó legfeljebb 25 üzenetet. „Sikertelen”-nél a sor hibaszövege mondja meg az okot; javítás után `Újraküldés` soronként, `Kijelöltek újraküldése` vagy `Minden sikertelen újraküldése` (megerősítéssel, az SMS újra költséggel jár); a bizalmas tartalmú (hivatkozás, kód, PIN) üzenet nem küldhető újra, az ügyfél újat kér. Ha maga a küldő feladat bukott el (adatbázis- vagy kódhiba), az oldal „Sikertelen feladatok” táblázata mutatja a hibával: `Újrafuttatás` vagy `Törlés`; a sor egészében a `Sor ürítése` gombbal üríthető (a várakozó üzeneteket az ütemező újrasorolja, a várakozó átvételt újra kell indítani).
2. Kapcsolatpróba az ügyfelek érintése nélkül: `docker compose exec web php artisan hdid:test-mail cim@example.org --now`, illetve `hdid:test-sms +36301234567 --now`. Az EWS hibaszövege a HTTP-státusszal kezdődik: `401` után a kiszolgáló által felkínált hitelesítési módok állnak (csak `NTLM` / `Negotiate`: `EWS_AUTH=ntlm` tartományi felhasználónévvel, az image 1.8.0-től NTLM-képes libcurl-lel jön; `Basic` mellett jelszó és postaláda; Exchange Online-nál `oauth`), `404` vagy „HTML answer” rossz `EWS_URL`-t (`https://…/EWS/Exchange.asmx`) vagy közbeékelődő bejelentkező oldalt jelent, az `EWS Error…` kód és szöveg az Exchange saját válasza (postaláda, küldési jog), a „could not be reached” hálózati, proxy- vagy tanúsítványhiba.
3. Belső CA-val hitelesítő Exchange-nél a CA-fájl a `ca` mappába kerüljön, majd `docker compose restart web worker sync-worker scheduler` (a CA-kat a konténer induláskor olvassa be, a `docker compose up -d` változatlan beállításnál nem indít újra); a TLS-ellenőrzést ne kapcsolja ki.
4. „Nem kézbesíthető” SMS: az átjáró elfogadta, de a hálózat szerint nem ért célba; a hibamező az átjáró állapotszava, a rendszer nem küldi újra magától (ellenőrizze a számot, majd `Újraküldés`). „Elküldve · nincs jelentés”: a beállított lekérdezésekre nem jött jelentés; ha minden SMS így végzi, az `OZEKI_STATUS_ACTION` vagy az állapotszavak nem illenek az átjáróhoz (telepítési útmutató 11. fejezet). Nyomok: `message.delivered`, `message.undeliverable` az auditnaplóban, `SMS delivery status query failed` a `docker compose logs scheduler` kimenetében.
5. A kimenő tűzfal engedje a konténerek felől az EWS/SMTP és az Ozeki címét.
6. **SMTP STARTTLS nélkül** (piros Levelezés kártya, „TLS required”): a kiszolgáló nem kínál titkosítást, a `MAIL_REQUIRE_TLS` pedig alapból kötelezővé teszi. Teendő: STARTTLS a kiszolgálón, vagy `MAIL_SCHEME=smtps` a 465-ös porttal; `MAIL_REQUIRE_TLS=false` csak titkosítás nélküli helyi próbakiszolgálóhoz. EWS-en a levelek mentett másolat nélkül mennek; ha egy belépési levél mégis a küldő postafiókban marad, azt az Exchange saját naplózása vagy archiválása tartotta meg.
7. **Visszavont SMS:** SMS csak az elfogadott országok mobilszámára megy (*Beállítások › Azonosítás › Telefonszámok*); más számra az üzenet visszavont, az ok az üzenetnél (`message.refused sms_destination`), a `hdid:test-sms` „cancelled · Not sent: …” kimenettel és nem nulla kóddal tér vissza. A várakozás közben lejárt vagy újabbal felváltott belépési kód, illetve hivatkozás is visszavont („lejárt, mielőtt kiküldhető lett volna”, „egy újabb váltotta fel”).
8. **Belépési plafon:** rendszerszinten óránként legfeljebb 10 000 belépési hivatkozás (0 = nincs plafon) és óránként 60, naponta 300 SMS-belépőkód megy ki, egy számra naponta 5 (*Beállítások › Ügyfélportál › Belépési módok*). Betelt plafonnál nem megy ki üzenet, a portál belépő oldala ezt kiírja, az audit `login.request_capped`. Áradatra utal; a plafont csak valódi belépési csúcsnál emelje, a hivatkozását a levelezőszolgáltató valódi korlátjához igazítsa (például egy Exchange Online-postafiókról percenként 30 levél, 24 óra alatt 10 000 címzett).

9. **Elsőbbségi sor:** a belépési kódok a `priority` soron mennek; a régi, `command: [worker, default]` parancsú `compose.yaml` ezt nem hallgatja, ilyenkor a kódok nem mennek ki (az Üzenetsorok oldal „priority: még nem látott feldolgozót” jelez): az új `compose.yaml` letöltése (UPGRADE.md), vagy a `HDID_PRIORITY_QUEUE` üresen hagyása.

Az ügyféltörzs-átvétel hibát jelez

Az *Ügyféltörzs-átvételek* lista hibaszövege dönt; az okok részletes listája a Rendszerállapot kártyájának tanácsában olvasható.

- **„Üres forrás” és „Gyanús forrás”:** a védőkorlát megállította a futást, senki nem zárult le. Ellenőrizze az export teljességét.
- **Hitelesítési hiba:** `docker compose exec web php artisan hdid:emd-sync-token`.
- **„Hibás belépési válasz”:** A belépés `POST` az `EMD_SYNC_LOGIN_URL` címre, JSON-törzzsel: `{"Password": "…", "Username": "…"}`; a rendszer 2xx választ vár, a törzs legfelső szintjén nem üres `access_token` és `refresh_token` mezővel. „Hibás belépési válasznál” a hibaszöveg a HTTP-státuszt és a kapott mezők nevét mondja (értékek nélkül), vagy azt, hogy a válasz nem JSON. Kézi próba ugyanazzal a kéréssel: megmutatja, hogyan érkezett meg a felhasználónév (JSON-ban egy backslash `\\` alakban látszik) és a jelszó (hossza, `=`-re végződik-e, van-e benne szóköz vagy idézőjel), majd a válasz státuszát és szerkezetét a tokenek értéke nélkül:

```
docker compose exec web php artisan tinker --execute '$u =
config("hdid.sync.username"); $p = (string) config("hdid.sync.password"); echo
"Username: ".json_encode($u)."
Password: ".strlen($p)." karakter; =-re végződik: ".(str_ends_with($p, "=") ? "igen" :
"nem")."; szóköz vagy idézőjel benne: ".(preg_match("/[\s\"']/", $p) ? "IGEN" :
"nem")."
Kérés: POST ".config("hdid.sync.login_url")."
  ".json_encode(["Password" => "***", "Username" => $u], JSON_UNESCAPED_UNICODE)."
"; $r = Illuminate\Support\Facades\Http::acceptJson()->asJson()->withoutRedirecting()-
>timeout(20)->post(config("hdid.sync.login_url"), ["Password" => $p, "Username" =>
$u]); echo "Válasz: HTTP ".$r->status()."
"; $walk = function ($v, $path) use (&$walk) { if (is_array($v)) { foreach ($v as $k =>
$x) { $walk($x, $path === "" ? (string) $k : "$path.$k"); } return; } echo "  $path:
".gettype($v).(is_string($v) ? " (".strlen($v)." karakter)" : "")."
"; }; $j = $r->json(); if (is_array($j)) { $walk($j, ""); } else { echo "  nem JSON:
".mb_substr(trim(strip_tags($r->body())), 0, 200)."
"; }'
```

- **„Nem tudott kapcsolódni”** (belépés, ID-lista vagy export): a hibaszöveg zárójelben megnevezi a kiszolgálót és az okot. A kérés adatai nem kerülnek bele.
  - *a név a konténerből nem oldható fel (DNS)*: a konténer a gazdagép DNS-kiszolgálóit kérdezi, a gazdagép `/etc/hosts`-át nem látja. Csak ott szereplő névhez `extra_hosts` kell a `compose.override.yaml`-ban; helyi feloldós vagy VPN-es gazdagépnél a belső DNS-t a `/etc/docker/daemon.json` `dns` kulcsa adja (telepítési útmutató 6.7);
  - *a kapcsolatot elutasították, vagy a cím nem érhető el*: tűzfal a Docker-hálózat (a `docker network inspect openhdid_default` szerinti alhálózat) és a kiszolgáló között, útválasztás vagy rossz port;
  - *nem jött válasz időben*: jellemzően csendben eldobó tűzfal;

- *a tanúsítvány nem ellenőrizhető* (OpenSSL 20, „unable to get local issuer certificate”): a kiszolgáló tanúsítványát belső tanúsítványkiadó írta alá. A lánc a gazdagépről:

  ```
  openssl s_client -connect emd.ceg.local:443 -servername emd.ceg.local -showcerts
  </dev/null 2>/dev/null | grep -E '^ *[0-9]+ s:|^ *i:'
  ```

  Az utolsó `i:` sor a hiányzó kiadó (jellemzően a vállalati gyökér-CA). A tanúsítványa PEM-formátumban (`-----BEGIN CERTIFICATE-----` ; Windows-ból „Base-64 kódolású X.509”, DER-ből `openssl x509 -inform der -in ca.cer -out ca.crt`) a `ca` mappába kerül `.crt` vagy `.pem` kiterjesztéssel, mindenki számára olvashatóan (644); ha a kiszolgáló a köztes tanúsítványt sem küldi, az is. Ha a gazdagép maga már megbízik benne (a `curl https://emd.ceg.local/` a gazdagépről hiba nélkül fut), a gazdagép tanúsítványtárában (`/etc/pki/ca-trust/source/anchors/`) megvan. Utána `docker compose restart web worker sync-worker scheduler` ; a `docker compose logs web | grep "extra CA"` mutatja, hány CA-fájlt vett fel.

A konténer szemszögéből minden beállított ügyféltörzs-címet egyszerre ellenőriz ez a parancs (DNS, TCP és HTTPS, a nyers hibaüzenettel):

```
docker compose exec web php artisan tinker --execute 'foreach
(["login_url","refresh_url","id_list_url","api_url"] as $k) { $u =
config("hdid.sync.$k"); if (! $u) { echo "$k: nincs beállítva
"; continue; } $h = parse_url($u, PHP_URL_HOST); $p = parse_url($u, PHP_URL_PORT) ?:
(parse_url($u, PHP_URL_SCHEME) === "http" ? 80 : 443); $ip = filter_var($h,
FILTER_VALIDATE_IP) ? $h : gethostbyname($h); echo "$k: $h:$p
  DNS:   ".($ip === $h && ! filter_var($h, FILTER_VALIDATE_IP) ? "NEM OLDÓDIK FEL" :
$ip)."
"; $s = @fsockopen($ip, $p, $en, $es, 5); echo "  TCP:   ".($s ? "kapcsolódik" : "HIBA
($es)")."
"; try { $r = Illuminate\Support\Facades\Http::timeout(10)->connectTimeout(5)->get($u);
echo "  HTTPS: válaszol (HTTP ".$r->status().")
"; } catch (Throwable $e) { echo "  HTTPS: ".preg_replace("/ \(see .*\$/s", "", $e-
>getMessage())."
"; } }'
```

- **Próbafuttatás javítás után:** `docker compose exec web php artisan hdid:emd-sync --dry-run` .
- **Túl régi utolsó sikeres futás:** `docker compose exec scheduler php artisan schedule:list` mutatja, ütemezve van-e a `hdid:emd-sync` (`EMD_SYNC_CRON`).
- **E-mail-ütközés:** a Rendszerállapot sárga, a futás részleteinek „E-mail-ütközések” blokkja mutatja, kinek a címét nem lehetett átírni, mert másik fiókon áll; a fiók a régi címmel marad, és nem tud belépni, amíg az ütközést az ügyféltörzsben nem rendezik, vagy a másik fiókot nem törlik. A naplóban (`docker compose logs sync-worker`) figyelmeztető sor, a `hdid:emd-sync` kimenete is kiírja az okot. Az „Eset” oszlop: átnevezés egy másik fiók címére; új tétel, amelynek címe már egy másik fiókon áll, esetleg csak írásmódban (ékezet, kis- és nagybetű, teljes szélességű karakter, hasonló alakú domain) eltérve: a meglévő fiók tovább működik, nem kell törölni, a tétel címét kell javítani az ügyféltörzsben; demótétel, élesben nem kerül átadásra.
- **Más személyhez került fiókok és újra kiadott tételek:** a futás részleteinek „Más személyhez került fiókok” blokkja azokat a fiókokat sorolja, amelyeket az átvétel másik tételhez kötött át úgy, hogy a régi tétel ugyanabban a futásban még jelen volt (két élő tétel, két személy), vagy kézi címcsere után először kötött tételhez: a fiók mindenhonnan kilépett, és törlődött, amit az előző birtokos beállított

(munkatársnál jelszó, második lépés, belépési kötés; ügyfélnél PIN, válaszok, kézi számok). Ha a régi tétel már nincs a forrásban, az ugyanannak a személynek az újrakiadása új külső azonosítóval: az „Ugyanannak a személynek újra kiadott tételek” blokk mutatja, semmi nem törlődik, csak a belépések szűntek meg (új belépési hivatkozás vagy kód kell); az auditnaplóban `account.reissued`. Az ismert tétel, amelynek új címe nem használható e-mail-cím, a régi címén marad, és nem zárul le.

- **Besorolásváltás (ügyfél és munkatárs között):** a régi fiók csak annyi egymást követő futás után zárul le, mint egy hiányzó tétel; a függőben lévők a futás részletein látszanak. Megszűnő főügyfél-jogosultságnál a kapcsolatok magától megszűnnek, a kapcsolt ügyfelek nem zárulnak le.

A naplóban nem a kliensek IP-címe látszik

Ha az access log `ip` mezője minden kérésnél a Docker-átjáró vagy a terheléselosztó címe, az IP-alapú forgalomkorlát és az auditnapló címe hamis. A lehetséges okok és teendők:

- **IPv6-kapcsolat:** kapcsolja be a Docker IPv6-NAT-ját (`ip6tables`).
- **Terheléselosztó mögött:** `OPENHDID_TLS=off` és a `TRUSTED_PROXIES` a terheléselosztó címe (telepítési útmutató 6.5). Hamis cím mellett a kliens-címenkénti kéréskorlát is minden kérést egy címnek számol (429).
- **Userland-proxy:** kapcsolja ki a Docker-démonban (`"userland-proxy": false`).
- **Rootless Docker:** az alapértelmezett `builtin` port-továbbító elveszti a forrás-IP-t; a démon felhasználói egységében `DOCKERD_ROOTLESS_ROOTLESSKIT_PORT_DRIVER=slirp4netns` (telepítési útmutató 3. fejezet, „Rootless Docker”), vagy TLS-t lezáró előtét `OPENHDID_TLS=off` és `TRUSTED_PROXIES` beállítással.

Minden oldal 400 „Ismeretlen kiszolgálónév” választ ad

Az alkalmazás csak az `APP_URL` gépnevére, a `localhost` névre (a konténer életjel-próbája) és az `OPENHDID_EXTRA_HOSTS` neveire szóló kérést fogadja. Ok: az `APP_URL` gépneve eltér a böngésző címétől, egy előtét más `Host` vagy `X-Forwarded-Host` fejlécet küld, vagy egy gépi partner neve hiányzik az `OPENHDID_EXTRA_HOSTS` listából. Javítás a `.env`-ben, majd `docker compose up -d`. Az `OPENHDID_EXTRA_HOSTS` nevei gépi hívásra valók: böngészőből a panel és a portál csak az `APP_URL` címén működik.

429 vagy 503 válaszok

- **429 „Túl sok kérés érkezett erről a címről…”** (`Retry-After: 5`): az nginx kliens-címenkénti korlátja (alapból 20 kérés/s, 100-as löket, 16 egyidejű kérés; IPv6-on a /64-es hálózat a kulcs; a `HDID_RATE_LIMIT_EXEMPT_NETWORKS` címeire nem hat). A naplóban `limiting requests`, illetve `limiting connections` sor. Ha sok valódi felhasználó egy címről érkezik (vállalati NAT), a `HDID_RATE_LIMIT_EXEMPT_NETWORKS` a megoldás; ha minden kérés egy címről látszik (rootless Docker, terheléselosztó `TRUSTED_PROXIES` nélkül, újratitkosító előtét `OPENHDID_TLS=on` mellett), előbb a valódi kliens-címet kell helyreállítani; végső esetben az `OPENHDID_NGINX_RATE_LIMIT` és `_CONN_LIMIT` emelése vagy `0`, majd `docker compose up -d`. A belépési útvonalak saját, alkalmazásszintű korlátja (percenként 5 kérés címenként) JSON 429-et ad.
- **503 „A szolgáltatás most túlterhelt…”** (`Retry-After: 10`): nincs szabad PHP-FPM folyamat, és a várakozó sor (`OPENHDID_FPM_BACKLOG`) is tele. Tartósan: a *Kapacitás* kártya és a 9. fejezet (több folyamat, nagyobb `OPENHDID_WEB_MEM_LIMIT`, a következő profil); rövid csúcsnál a rendszer

szándékosan utasít el, hogy ne omoljon össze. A térfogati (hálózati szintű) DDoS ellen a szolgáltató vagy egy előtét véd.

Valkey: memória, munkamenet-tár, közös tár

1. **Kiszorító politika a fő Valkey-n** (piros): külső vagy menedzselt Valkey-n `maxmemory-policy noeviction` kell (a menedzselt alapérték gyakran `volatile-lru`); a beépített így fut.
2. **80 % fölött:** `docker compose exec valkey valkey-cli -a "$REDIS_PASSWORD" INFO memory` (a `used_memory_human` és a `maxmemory_human` sor), és az `OPENHDID_VALKEY_MAXMEMORY`, `OPENHDID_VALKEY_MEM_LIMIT` emelése, majd `docker compose up -d`; a tár jellemzően a várakozó üzenetektől nő. Süti nélküli vendégkérés munkamenetet nem nyit; a panel belépőoldala, a `/sanctum/csrf-cookie`, a Livewire és az SSO-indítás igen (kb. 0,85 KB), ezek a munkamenet-Valkey-be kerülnek.
3. **Munkamenet-tár sárga:** a munkamenetek a fő Valkey-ben vannak (külső Valkey-nél nincs `REDIS_SESSION_HOST`: második példány kell `allkeys-lru`-val), vagy az elmúlt órában munkamenetek szorultak ki (áradat, vagy kicsi az `OPENHDID_VALKEY_SESSIONS_MAXMEMORY`). Kiszorításkor a régóta tétlen felhasználók újra belépnek; a belépés, a sor és a zárak közben működnek. Piros: a munkamenet-tár nem érhető el, senki nem tud belépni.
4. **Másik telepítés ugyanazon a tárón** (az ütemező és a sor kártyája piros, az Üzenetsorok oldal figyelmeztet): két telepítés (például éles és teszt) ugyanazt a Valkey-t és kulcselőtagot használja. A telepítésazonosító (12 hexa karakter az `APP_KEY`-ből és az `APP_URL` normalizált alakjából) a zárakat, az ütemező mutexeit és a beállítások gyorsítótárát elválasztja, így mindkét telepítés lefuttatja a saját feladatait és migrációját; a várakozó feladatok, az életjelek és a korlátszámlálók viszont közösek maradnak. Teendő: saját Valkey (a mérő- és tesztpéldánynak is), vagy legalább saját adatbázis, ACL-felhasználó és `REDIS_PREFIX` / `CACHE_PREFIX`. A jelzés 15 perc után cseng le; `APP_KEY`- vagy `APP_URL`-csere után legfeljebb egy percig az ütemező kártyája is piros lehet.
5. **Előtagcsere:** `docker compose stop web scheduler`; a feldolgozók kiürítik a sort (`docker compose exec worker php artisan queue:monitor redis:priority,redis:default,redis:sync`, vagy az Üzenetsorok oldal); a `REDIS_PREFIX` és `CACHE_PREFIX` cseréje a `.env`-ben; `docker compose up -d`. Minden munkamenet véget ér.

Demóadat éles környezetben

Éles környezetben (`APP_ENV=production`) a `migrate` 64-es kóddal, a talált címeket megnevező hibával megáll, amíg az adatbázisban demóadat van (`.test` tartományú fiókok és ügyféltörzstételek, a törölt sorok is, demóhívások), ezért a `web`, a `worker` és az ütemező sem indul el; futó rendszeren a *Demóadatok* kártya piros, a demófiókok nem léphetnek be (`login.rejected`, `demo_account`), és az átvétel a demótételt nem adja át a valódi személynek (`sync.demo_conflict`). Ha egy demópéldányt csak az `APP_ENV` átírásával vitt élesbe, üres adatbázissal kell újrakezdeni: `docker compose down -v` (a kötetek, a feltöltött fájlokkal együtt törlődnek), majd `docker compose up -d` és `hdid:make-admin`. Ha csak néhány valódi fióknak van `.test` címe (téves felismerés), nem kell törölni: átmenetileg `APP_ENV=staging`, a címek javítása, majd vissza `production`.

Vállalati hálózat a korlátokba ütközik

Ha sok felhasználó egy közös címről (NAT) vagy IPv6-on egy közös /64-es hálózatból (egy vállalati LAN) érkezik, a kliens-címenkénti korlátok (az nginx kérés- és kapcsolatkorlátja, a belépési útvonalak percenkénti 5 kérése, a panel belépőjének 5 próbája) a valódi forgalmat is elutasíthatják. A `.env` `HDID_RATE_LIMIT_EXEMPT_NETWORKS` sora (CIDR, elemenként legfeljebb /8, illetve /32) mentesíti a

hálózatot, majd `docker compose up -d`; a konténer induláskor kiírja a mentes hálózatokat, a Rendszerállapot alsó blokkja is mutatja. Az e-mail-címenkénti és számonkénti korlátok, a belépési plafonok, a fiókzárolás és a gépi kulcsok korlátjai továbbra is hatnak. Terheléselosztó mögött csak `OPENHDID_TLS=off` és helyes `TRUSTED_PROXIES` mellett hat; az elosztó címét felvenni tilos, mert az a teljes internetet mentesítené.

Szerepkörök ellenőrzése a 2026. októberi frissítés után

A *Szerepkörök és jogosultságok* oldalhoz korábban nem volt bekötve jogosultsági szabályzat, így bármely munkatárs szerkeszthette a szerepköröket (minden korábbi verziót érintett). A frissítés után nézze át az auditnaplóban a `role.updated` és `role.created` eseményeket és a szerepkörök jogkészletét. Ha a főrendszergazda szerepkört átnevezték, a `docker compose exec web php artisan hdid:make-admin <cím>` helyreállítja (újra létrehozza a teljes jogkészlettel); a régi, átnevezett szerepkör egyéni szerepkörként a tagjaival megmarad: ellenőrizze, és ossza újra a tagokat. Ettől a kiadástól szerepkört csak az oszthat ki, akinek a szerepkör minden joga megvan, és csak olyan fiók módosítható, amelynek minden szerepkörét a kezelő kioszthatná (`user.role_escalation_rejected`, `user.manage_rejected`).

Kiszolgáló kompromittálódása

1. **Elszigetelés:** izolálja a gazdagépet, és őrizze meg a naplókat.
2. **Új gép:** állítson fel tiszta gazdagépet, és a telepítést ellenőrzött kiadásból (aláírt image, 8. fejezet) és mentésből állítsa vissza.
3. **Titokcsere:** cserélje az összes titkot: adatbázis, EWS, Ozeki, SSO, API-kulcsok, ügyféltörzs, TLS-kulcs.
4. **Visszavonások:** vonja vissza a munkameneteket és a tokeneket. Az `APP_KEY` cseréje csak külön terv szerint történhet (5. fejezet).

---

7 Mentés és visszaállítás

A kiadás `backup.sh` szkriptje egy paranccsal ment és állít vissza. A telepítési mappában, a `compose.yaml` mellett él, bárhonnan indítható, és root nélkül fut: sem `sudo`, sem root jogú konténer nem kell, rootful és rootless Dockerrel egyaránt működik. Egy mentés egyszerre fogja össze azt, ami a helyreállításhoz kell:

1. **az adatbázist** (a beépített MariaDB-ét, konzisztens pillanatképként);
2. **a `storage` kötetet** (arculati képek, feltöltések);
3. **a beállításokat:** `.env`, `db.env` (benne az `APP_KEY` és minden jelszó), `compose.yaml`, `compose.override.yaml`, a `ca` mappa;
4. **a `tls` mappát** (tanúsítvány és kulcs).

```
cd /opt/openhdid
./backup.sh                          # mentés: backup/2026-10-09_023000
./backup.sh list                     # a mentések listája (verzió, adatbázis, méret)
./backup.sh restore                  # visszaállítás a legutóbbi mentésből
./backup.sh restore 2026-10-09_023000
```

A mentés. A szkript a saját mappájában létrehozza a `backup` almappát (700-as jog), és abba egy időbélyeges mappát: `database.sql.gz`, `storage.tar.gz`, `config.tar.gz`, `tls.tar.gz`, `info.txt` (időpont, OpenHDID-verzió, image) és `SHA256SUMS`; minden fájl 600-as jogú. A mentés futó vermen készül (a `mariadb` szolgáltatásnak futnia kell), kiesés nélkül. Alapból a legutóbbi 14 mentés marad, a régebbieket a szkript törli (`--keep N`, `0` = mind marad); a visszaállítás előtti biztonsági mentésekből a legutóbbi 3, mert azok is tartalmazzák a kulcsokat és az adatokat. Egy megszakadt futás (újraindítás, `kill -9`) zárját a következő futás felismeri és feloldja. A szkript a telepítési mappa `.env`-jét követi: a héjban exportált `COMPOSE_PROJECT_NAME`, `COMPOSE_FILE` és `COMPOSE_PROFILES` nem hat rá (két telepítésnél egy gépen mindegyik `.env`-jébe saját `COMPOSE_PROJECT_NAME` kerül). A `tls` kulcsát az alkalmazás saját felhasználója olvassa ki, mert az üzemeltető számára nem olvasható.

Ütemezve (cron, naponta 2:30-kor; a `--quiet` csak hibánál ír, a cron ezt levélben küldi):

```
30 2 * * * /opt/openhdid/backup.sh --quiet
```

Visszaállítás

A `./backup.sh restore [név]` egy megerősítés (`igen`) után végigviszi a teljes helyreállítást; nem interaktív futtatáshoz `--yes`. A lépések:

1. **Ellenőrzés:** a mentés ellenőrzőösszegei (ezek a sérülést mutatják ki, a szándékos módosítást nem: aki a mentést átírja, az összegeket is átírhatja); a beállítás-archívumban csak a várt fájlok lehetnek (`.env`, `db.env`, `compose.yaml`, `compose.override.yaml`, `ca`), hivatkozás nélkül; adatbázist tartalmazó mentésnél a mentett beállításokban benne a `db` profil; a `tls` mappa az üzemeltetőé. Bármelyik hiba esetén semmi nem változik, az alkalmazás fut tovább. Ha a mentés beállításai eltérnek a jelenlegiektől, a szkript a megerősítés előtt megnevezi a fájlokat.
2. **Biztonsági mentés** a jelenlegi állapotról (`…_visszaallitas-elott`, a legutóbbi 3 marad), így a visszaállítás is visszavonható; ha bármelyik lépése hibázik, a visszaállítás nem kezdődik el. Ha a jelenlegi verem már nem menthető, `--no-safety-backup`.
3. **Az alkalmazás leáll** (`web`, `worker`, `sync-worker`, `scheduler`); innentől a végéig kiesés van; ha nem áll le, a visszaállítás nem kezdődik el.
4. **Beállítások és `tls`:** a mentett `.env` és a többi fájl kerül vissza, a mentés `OPENHDID_VERSION`-jével együtt; a kulcsot az alkalmazás felhasználója a 700-as `backup` mappán belül egy ideiglenes mappába írja, amely aztán a `tls` helyére kerül, így más helyi felhasználó egyik pillanatban sem fér hozzá.
5. **Adatbázis:** a táblák törlése, majd a mentés visszatöltése a migrációs fiókkal, sandbox módban (a mentés adat: a benne lévő kliensparancsok nem futnak).
6. **Kiléptetés:** minden munkatárs és ügyfél munkamenet-korszaka eggyel nő, az „emlékezz rám” és a mobil tokenek törlődnek, az élő egyszeri titkok (belépési hivatkozás, kódok) érvényüket vesztik, a munkamenet-tár és a gyorsítótár ürül az adatbázisban, a `storage` kötetben és a Valkey-ben is (`FLUSHDB` a `REDIS_DB`, a `REDIS_CACHE_DB` és a munkamenet-Valkey adatbázisán). Így a mentés óta visszavont belépések (kilépés minden eszközről, lezárt fiók) nem élednek újra: mindenkinek újra be kell lépnie, a mobilalkalmazásban is. Bejegyzés: `access.revoked_all` (`reason=backup_restore`, darabszámokkal). A lépés az alkalmazás saját parancsa (`php artisan hdid:sign-out-all --force --reason=backup_restore`) a web egyszeri konténerében, root nélkül, az alkalmazás beállításaival, így külső adatbázison és Valkey-n is; ugyanez a parancs futtatható kézzel is (12. fejezet). A Valkey-ben várakozó sorfeladatok is elvesznek: a ki nem küldött üzeneteket az ütemező öt percen belül újrasorolja (a küldés közben megszakadtakat 30 perc után), egy

várakozó ügyféltörzs-átvételt újra kell indítani; a visszajátszás-védelem és a korlátszámlálók is nullázódnak, a kérdés-válasz kézi feloldásai elvesznek. Közös adatbázisszám mellett egy másik telepítés munkamenetei is törlődnének: egy Valkey-adatbázis egy telepítésé, a menedzselt Valkey engedje a `FLUSHDB`-t. Ha a kiléptetés elbukik, az alkalmazás leállítva marad: a hiba elhárítása után `./backup.sh signout`, majd `docker compose up -d`.

7. **Storage kötet**, majd **indítás** és rendszerállapot.

Kiléptetés külön. A `./backup.sh signout [--yes]` ugyanezt a kiléptetést végzi visszaállítás nélkül, rövid leállással (megerősítést kér); kell például, ha egy külső adatbázist csak a szkript után állítanak vissza. A mentés óta tett biztonsági lépéseket (fióklezárás, API-kulcs visszavonása, szerepkör- és jogelvétel, jelszó- és második lépés-visszaállítás) a visszaállítás után meg kell ismételni.

Ellenőrzés utána: belépés; egy ügyfél adatlapján a válaszok száma és egy kérdés-válasz próba. Ez bizonyítja, hogy a titkosított adatok az `APP_KEY`-jel olvashatók.

Új gépre. Docker telepítése (telepítési útmutató 2. fejezet), a telepítési mappa létrehozása, benne a kiadás `compose.yaml` és `backup.sh` fájlja és a `tls`, `ca` mappa (`mkdir -p tls ca`), majd a mentés mappájának átmásolása a `backup` alá, végül `./backup.sh restore <név>`. A beépített adatbázist és a két fiókot az első induló `mariadb` a visszaállított jelszavakkal hozza létre.

Követelmények a mentéssel szemben.

- A `backup` mappát rendszeresen másolja a gazdagépen kívülre, titkosítva és módosítás ellen védve; a helyi mentés a gép elvesztése ellen nem véd. Csak olyan helyről állítson vissza, ahová illetéktelen nem írhat: a mentés az indítási beállításokat (`compose.yaml`, `.env`) is tartalmazza, így egy módosított mentés azt is meghatározná, mi fut a gépen.
- A `config.tar.gz` (kulcsok, jelszavak) más hozzáférési szabállyal kerüljön tárolásra, mint az adatbázis.
- Határozza meg a megengedett adatvesztést (RPO) és a helyreállítási időt (RTO), és rendszeres visszaállítási próbával mérje; a próba egy másik gépen a fenti „Új gépre” lépésekkel végezhető.
- Külső adatbázisnál (a `db` profil nélkül) az adatbázist az adatbázis üzemeltetője menti és állítja vissza, a mentéssel egyidejű állapotra; a szkript a többit kezeli, és ezt az `info.txt`-ben jelzi (`database=external`). Ha az adatbázis a szkript után áll vissza, utána `./backup.sh signout`.
- **Két alkalmazásgép (10k):** a `storage` kötet NFS-en közös, ezért teljes mentést elég az A gépen futtatni; a B gép saját `.env`-jét, `db.env`-jét és `tls` mappáját ott kell menteni (a `./backup.sh` ott is futhat, a `storage`-ot másodszor is elmenti, ez nem árt). Visszaállítás: előbb a B gépen `docker compose stop` (a szkript csak a helyi gépet állítja le, a másik közben írná az NFS-t és az adatbázist); az adatbázist az üzemeltetője állítja vissza; utána az A gépen `./backup.sh restore <név>`, majd `./backup.sh signout`, ha az adatbázis a szkript után állt vissza; végül a B gépen `docker compose up -d`. A kiléptetés a közös Valkey adatbázisait üríti, mindkét gépet érinti.

---

8 Frissítés és visszaállás

Kiadások. A kiadások a GitHub Releases oldalon jelennek meg, jegyzettel, dokumentumokkal és ellenőrzőösszegekkel.

- **Verziózás:** SemVer. A javítóverzió (1.0.*x*) hibajavítást vagy platformfrissítést hoz (PHP, nginx, OpenSSL, Alpine), az alverzió (1.*x*) új funkciót. Ha a főverzió változik, a jegyzet leírja a teendőket.

- **Címkék:** egy kiadott címke soha nem változik.

1. **Az image ellenőrzése** (választható, ajánlott):

   ```
   gh attestation verify oci://ghcr.io/adamfhun/openhdid:1.14.0 --owner adamfhun
   cosign verify ghcr.io/adamfhun/openhdid:1.14.0 \
     --certificate-identity-regexp
   '^https://github.com/adamfhun/openhdid/\.github/workflows/release\.yml@refs/tags/v' \
     --certificate-oidc-issuer https://token.actions.githubusercontent.com
   ```

   Az aláírás ellenőrzéséhez cosign 3 vagy újabb kell: az image-ek a Sigstore-csomag formátumban vannak aláírva, ezt a cosign 2 nem keresi, és „no signatures found” hibát ad.
2. **Mentés:** `./backup.sh` (7. fejezet).
3. **Frissítés:** az `OPENHDID_VERSION` átírása, majd `docker compose pull && docker compose up -d`. A migrációk automatikusan lefutnak, a szolgáltatások az új image-dzsel indulnak.
4. **Ellenőrzés:** `docker compose ps`, Rendszerállapot (a fejlécben az új verzió), a telepítési útmutató 12. fejezetének próbái.

Két alkalmazásgép (10k) és Kubernetes (20k): a kiadás `UPGRADE.md` mellékletének „Két alkalmazásgép (10k profil)” és „Kubernetes (20k profil)” része: mentés a migráció előtt (a külső adatbázis a kiszolgáló saját eszközével), a B gép, ellenőrzés, az A gép; Kubernetesen a migrációs Job előbb, utána a gördülő frissítés. Ugyanott a két verzió együttfutásának feltétele és a visszalépés.

Visszaállás sikertelen frissítés után

1. **Ha nem futott migráció:** az `OPENHDID_VERSION` visszaírása az előző verzióra, majd `docker compose up -d`.
2. **Ha futott migráció:** a migrációk csak előre futnak. Állítsa vissza a frissítés előtti mentést: `./backup.sh restore <név>` (7. fejezet); ez a mentett `.env`-vel az előző verziót is visszahozza, mindenkit kiléptet, és elindít. A mentés óta keletkezett adatok elvesznek. Külső adatbázisnál az adatbázis visszaállítása után `./backup.sh signout`.

---

9 Skálázás és teljesítmény

A telepítés három használati profil egyikére méretezett (`5k` , `10k` , `20k` ; telepítési útmutató 15. fejezet). A profil adja a PHP-FPM folyamatszámot, a feldolgozók számát, az adatbázis és a két Valkey méretét és minden szolgáltatás memóriaplafonját, és a nevét a Rendszerállapot *Kapacitás* kártyája mutatja. A terhelést nem az ügyfelek száma hajtja, hanem két kérésfolyam: a portál egyidejű használói (az ügyfelek 5–8 %-a) és az ügyintézők vezérlőpultja (20–30 ügyintéző, ötmásodpercenkénti frissítés); a hívásesemények, az automata menü lekérdezései és az azonosítás ezek mellett elhanyagolhatók.

A méretezés elve. A méretek a normál csúcsterhelés kétszeresére szólnak (normál csúcs: profil nélkül kb. 25, 5k kb. 50, 10k kb. 75, 20k kb. 130 kérés/s): a normál csúcson legfeljebb kb. 50 % a kihasználtság, a másik fele a korlátokon (nginx- és ingress-korlát, útvonal- és azonosítónkénti korlátok) belül maradó rosszindulatú terhelés tartaléka. Ami ezen felül érkezik, azt a rendszer 429-cel vagy 503-mal elutasítja, nem omlik össze. A térfogati (hálózati szintű, L3/L4) DDoS nem az alkalmazás feladata: a szolgáltató DDoS-szűrése vagy egy előtét (terheléselosztó, WAF) védi ki.

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
Kipróbálás (profil nélkül, Compose vagy hagyományos)	Egy gép: web, feldolgozó, ügyféltörzs-feldolgozó, ütemező, MariaDB, két Valkey	1	2	8 GB	20 GB	8 PHP-FPM folyamat; a plafonok összege kb. 6,2 GB + 1 GB OS
5k (legfeljebb 5 000 ügyfél)	Egy gép: web, 2 feldolgozó, ügyféltörzs-feldolgozó, ütemező, MariaDB, fő Valkey és munkamenet-Valkey	1	4	10 GB	80 GB	12 PHP-FPM folyamat; MariaDB 1G puffer, 40 kapcsolat; fő Valkey 256mb (noeviction), munkamenet-Valkey 384mb (allkeys-lru); a plafonok összege kb. 8,8 GB + 1 GB OS
10k (Compose vagy hagyományos)	Alkalmazásgép terheléselosztó mögött (az elsődlegesen fut az ügyféltörzs-feldolgozó és az ütemező is)	2	4	6 GB	40 GB	gépenként 12 PHP-FPM folyamat; az elsődleges plafonjainak összege kb. 4,8 GB + 1 GB OS; egy gép kiesésekor a másik viszi a normál csúcsot
10k	Külső MariaDB, elsődleges + replika	2	2	8 GB	legalább 40 GB	innodb_buffer_pool_size 4G, max_connections 80 (a Kubernetes 10k-hoz is elég)
10k	Külső fő Valkey (sor, zárak, korlátok, gyorsítótár)	1	–	legalább 768 MB	–	maxmemory 256mb, maxmemory-policy noeviction (kötelező), AOF

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
10k	Külső munkamenet-Valkey (REDIS_SESSION_*)	1	–	legalább 1,25 GB	–	maxmemory 576mb, maxmemory-policy allkeys-lru (kötelező), RDB
10k	NFS a közös storage kötetnek	1	–	–	legalább 50 GB	arculati képek, feltöltések, auditarchívum
10k Kubernetes	Csomópontkészlet	3	4	8 GB	–	egy csomópont kiesésekor is elfér; MariaDB és a két Valkey a fenti 10k-sorok szerint (menedzselt)
10k Kubernetes	Podok (kérés = memóriakorlát; CPU-nak csak kérés van)	web 2–4 (HPA), feldolgozó 2, ügyféltörzs-feldolgozó 1, ütemező 1, migrate Job	web 1, feldolgozó 0,1, ügyféltörzs 0,25, ütemező 0,1, migrate 0,25	web 2 GiB, feldolgozó 384 MiB, ügyféltörzs 448 MiB, ütemező 1 GiB, migrate 384 MiB	RWX kötet 50 GB	web pod: 8 PHP-FPM folyamat, 2 nginx-dolgozó; ingress: 50 kérés/s, löket 200, 128 kapcsolat címenként
20k Kubernetes	Csomópontkészlet	3	4	12 GB	–	egy csomópont kiesésekor a HPA maximuma is elfér
20k Kubernetes	Podok (kérés = memóriakorlát)	web 3–6 (HPA), feldolgozó 3, ügyféltörzs-feldolgozó 1, ütemező 1, migrate Job	web 1, feldolgozó 0,1, ügyféltörzs 0,5, ütemező 0,1, migrate 0,25	web 2 GiB, feldolgozó 384 MiB, ügyféltörzs 448 MiB, ütemező 1 GiB, migrate 384 MiB	RWX kötet 50 GB	web pod: 8 PHP-FPM folyamat; a korlátok összege a HPA maximumán, frissítési többletpoddal kb. 17 GiB
20k	Menedzselt MariaDB	1 (+ replika)	4	16 GB	legalább 80 GB	innodb_buffer_pool_size 8G, max_connections 120
20k	Menedzselt fő Valkey	1	–	legalább 768 MB	–	maxmemory 256mb, noeviction (kötelező)

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
20k	Menedzselt munkamenet-Valkey	1	–	legalább 2,25 GB	–	maxmemory 1gb, allkeys-lru (kötelező)
Minden profil	Tartalék	–	–	–	–	A méretek a normál csúcs kétszeresére szólnak: a normál csúcson legfeljebb kb. 50 % a kihasználtság, a másik fele a korlátokon (nginx- és ingress-korlát, útvonal- és azonosítónkénti korlátok) belül maradó rosszindulatú terhelés tartaléka. Ami ezen felül érkezik, azt a rendszer 429-cel vagy 503-mal elutasítja, nem omlik össze.
Minden profil	Térfogati (hálózati szintű, L3/L4) DDoS	–	–	–	–	Nem az alkalmazás feladata: a szolgáltató DDoS-szűrése vagy egy előtét (terheléselosztó, WAF) védi ki.

A Kapacitás kártya

Minden fiók kérése legfeljebb ötpercenként egy időbélyeget hagy a fiókján (`last_seen_at` , a munkamenet-tárolótól függetlenül). A kártya ebből mutatja az utolsó 15 percben aktív fiókok számát, a mai csúcsot (a `/health` lekérdezései és az oldal megnyitásai frissítik), MariaDB-n a használt kapcsolatokat a `max_connections` értékhez képest, és a profil tervezett egyidejű felhasználószámát (a profil ügyfélszámának 8 %-a: 400, 800, 1 600). Sárga, ha a csúcs eléri a tervezett érték 80 %-át, vagy a kapcsolatok 80 %-a használatban van. Profil nélkül (`HDID_SIZING_PROFILE` üres) a számok csak tájékoztatnak.

Mikor és mire kell lépni

- **Sárga Kapacitás kártya:** a következő profil. 5k-ról 10k-ra: második alkalmazásgép, külső MariaDB és két külső Valkey, NFS (telepítési útmutató 15.4); 10k-ról 20k-ra: Kubernetes (15.5). A kártya akkor is sárga, ha a PHP-FPM folyamatok × memóriakorlát nem fér a `web` konténer plafonjába, ha kevés a `max_connections`, vagy ha kérések várnak szabad PHP-FPM folyamatra.
- **Sok várakozó üzenet a Feladatsor kártyán:** előbb egy további feldolgozó (`docker compose up -d --scale worker=3`; az üzenetek egyszer mennek ki több feldolgozó mellett is), aztán a profil.
- **Lassú vezérlőpult sok ügyintézőnél:** a számlálók közös gyorsítótárból jönnek (egy lekérdezés-sorozat frissítési időnként, nem ügyintézőnként), ezért a frissítési idő (Beállítások › Üzemeltetés és megőrzés) 5-ről 8–10 másodpercre emelése a lekérdezések számát csökkenti, a számlálókat nem késlelteti észrevehetően.
- **Rendelkezésre állás:** 5 000 ügyfél fölött a kétgépes 10k profil akkor is indokolt, ha a terhelés még nem kívánná.

Mérés

A helyes méret mérésből jön. Mielőtt profilt lép, mérje meg a jellemző terhelést a saját környezetében egy tesztpéldányon (például k6-tal), sohasem az éles rendszeren. A profil forgatókönyvei: egyidejű portálhasználók belépett munkamenettel (kezdőlap, saját adatok, kérdések, hírek, és a felhasználók ötöde az élő egyszeri kód négymásodpercenkénti lekérdezésével), 20–30 ügyintéző vezérlőpultja ötmásodpercenként, hívásesemények (érkezés, aktív, vége) és hívószám-felismerés a telefonközpont kulcsával. Küszöbök: a portál API p95 < 300 ms, a vezérlőpult p95 < 800 ms, a gépi API p95 < 200 ms, hiba < 1 %. A tesztpéldány saját Valkey-t kapjon (vagy legalább saját adatbázist, ACL-felhasználót és `REDIS_PREFIX`-et), és rajta az `OPENHDID_NGINX_RATE_LIMIT=0`, különben a mérőgép egyetlen címe a kéréskorlátba ütközik; a 429-es válaszokat a mérés külön számolja. A mérés alatt a Rendszerállapot *Kapacitás* kártyája mutatja az elért egyidejű számot; ha a küszöb a tervezett egyidejű felhasználószám elérése előtt sérül, a profil erőforrásait kell emelni (több PHP-FPM folyamat és memória, az adatbázis puffere), nem a profilt.

Számítási alap saját méretezéshez

- **Web:** PHP-FPM folyamat × (`OPENHDID_FPM_MEMORY_LIMIT` 128M + 22 MB) + 208 MB OPcache + nginx-dolgozók × 80 MB + `/tmp` + 64 MB ≤ `OPENHDID_WEB_MEM_LIMIT`; a foglalt folyamatok száma a terhelés × 50–80 ms faliidő (Little törvénye), a folyamatszám ennél nagyobb; `OPENHDID_FPM_PM=static` egyenletes terhelésnél.
- **CLI-folyamat** (feldolgozó, migrate): 256M + 22 MB + `/tmp`; az ütemező három folyamattal számol.
- **MariaDB:** puffer + 192 MB + `max_connections` × 20 MB ≤ `OPENHDID_DB_MEM_LIMIT`; `max_connections` ≥ PHP-FPM folyamat × webpéldányok a HPA maximumán (+1 frissítési pod) + feldolgozók + 1 + 3 + 6 tartalék.
- **Valkey:** mindhárom profilban két példány: a fő (gyorsítótár, sor, zárak, korlátszámlálók; `noeviction`, AOF-naplóval a sor tartósságáért) és a munkamenet-tár (`allkeys-lru`, RDB); plafon = 2 × `maxmemory` + 256 MB, illetve + 128 MB. A munkamenet-tár a normál csúcs × 0,85 KB × 7200 s vendégmunkamenetét és a valódi munkameneteket tartja, 20 % tartalékkal. A fő példány kiesése a várakozó feladatokat veszti el (a kimenő üzeneteket az ütemező percenként újrasorolja), a munkamenet-példányé a belépéseket.
- **Gép:** a plafonok összege + 1 GB OS ≤ RAM; Kubernetes-csomóponton 1,5 GB foglalt, és egy kiesett csomópont mellett is elfér.

- **Adatbázis-méret:** a legnagyobb táblák az auditnapló (a beállított keret, alapból 150 nap marad az adatbázisban, a többi archívumfájl; 11. fejezet), az azonosítási munkamenetek lépésekkel és a hívások; 20 000 ügyfélnél, napi 300 hívással 5 év alatt 5–10 GB.

Kubernetes-megfeleltetés

A 20k profil kész manifesztjei a kiadás `kubernetes` mappájában vannak (telepítési útmutató 15.5); a szerepek megfeleltetése:

SZEREP	KUBERNETES-ERŐFORRÁS
`web`	Deployment (gördülő frissítés, `maxUnavailable: 0`), HPA CPU alapján az overlay határai között, Service, Ingress (a TLS itt zárul le, `OPENHDID_TLS=off`), PodDisruptionBudget. Induló és készenléti próba: `exec openhdid healthcheck` (az FPM-en át); élő-próba: `openhdid healthcheck live` (csak az nginx, így egy terhelt pool miatt nem indul újra). Memóriakérés = korlát (2Gi), podonként 8 PHP-FPM folyamat; NetworkPolicy csak az ingress-vezérlőt engedi a 8080-as portra; az ingress címenként 50 kérés/s-ot, 4-szeres löketet és 128 kapcsolatot enged.
`worker`, `sync-worker`	Deployment; a `worker` replikaszáma a sorhossz szerint, a `sync-worker` egy példány `Recreate` stratégiával.
`scheduler`	Deployment 1 replikával, `Recreate` stratégiával.
`migrate`	Job a web frissítése előtt, a `db.env` migrációs fiókjával, a közös Valkey zárja alatt.
`storage`	ReadWriteMany PVC (NFS, CephFS, Azure Files, EFS).
titkok	Secret az `.env` és a `db.env` tartalmából (`envFrom`), a tanúsítvány TLS Secret; `*_FILE` változókkal fájlként csatolt titok is használható.
biztonsági környezet	`runAsNonRoot`, `runAsUser: 82`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `seccompProfile: RuntimeDefault`; `/tmp` memóriában lévő `emptyDir`.
MariaDB, Valkey	menedzselt szolgáltatás vagy operátor; egyetlen írható adatbázis-példány; két Valkey: a fő `noeviction`, a munkamenet-tár (`REDIS_SESSION_*`) `allkeys-lru` politikával.

Az Octane-szerű, memóriában maradó PHP-futtatás tovább növelhetné az áteresztőképességet, de az nginx + PHP-FPM modellt váltaná ki, és kódátvizsgálást igényel. Csak méréssel igazolt igény esetén jöhet szóba.

10 Biztonsági karbantartás

GYAKORISÁG	TEENDŐ
Folyamatosan	`/health`, konténerállapot, tanúsítványlejárat, kiugró 4xx/5xx, sikertelen belépések és kulcshasználat; a riasztás próbával igazolt felelőshöz érkezzen.
Hetente	A kiadott image-et a projekt hetente automatikusan átvizsgálja (Trivy). Ha javítható, magas kockázatú lelet van, javítóverzió készül, és a kiadási jegyzet jelzi. Kövesse a kiadásokat (GitHub „Watch › Releases”), és a gazdagép OS- és Docker-frissítéseit is.
Minden frissítéskor	Mentés, aláírás-ellenőrzés, a telepítési útmutató 12. fejezetének próbái.
Negyedévente	Hozzáférések, titkok és gépi kulcsok felülvizsgálata, mentés-visszaállítási próba, tűzfalszabályok, az OWASP-eltérések (telepítési útmutató 12.3) állapota.

Támogatási naptár (a 2026. szeptemberi állapot):

- PHP 8.5 aktív támogatás 2027. 12. 31-ig;
- Alpine 3.24 2028. 05.-ig;
- OpenSSL 3.5 LTS 2030-ig;
- MariaDB 12.3 LTS 2029. 06.-ig;
- a buildhez használt Node.js 2026. 10. 28-tól a 26 LTS ágra vált.

A projekt a lejárat előtt javítóverzióban lép a következő ágra, és a kiadási jegyzet jelzi.

Az image védelmei:

- nem root felhasználó, csak olvasható fájlrendszer, elvett Linux-képességek, `no-new-privileges`;
- digesttel rögzített alapimage-ek, aláírt image, SBOM.

A `compose.yaml` biztonsági beállításait (`read_only`, `cap_drop`, `security_opt`) ne lazítsa.

11 Adatmegőrzés

A megőrzést három felületi beállítás szabályozza (*Beállítások › Üzemeltetés és megőrzés*):

- **általános megőrzés** (alapból 5 év): azonosítási munkamenetek, hívások, kimenő üzenetek, átvételek és az auditarchívum fájljai;
- **rövid életű tételek** (alapból 2 év): lejárt egyszeri kódok, feltöltött állományok, az átvételek változásrekordjai (*Ügyféltörzs-változások*);
- **auditnapló az adatbázisban** (alapból 150 nap, legalább 30): ennyi napig marad egy auditbejegyzés az adatbázisban, ahol a panel keresi; a régebbi napokat az éjjeli feladat napi, tömörített JSON Lines fájlokba írja (`storage/app/private/audit-archive/ÉÉÉÉ/audit-ÉÉÉÉ-HH-NN.jsonl.gz`, mellette `.sha256` ellenőrzőfájl), és csak a sikeres írás után törli a táblából. A fájlok az *Adminisztráció ›*

Rendszer › Auditarchívum oldalon listázhatók és letölthetők (minden letöltés `audit.archive_downloaded` esemény), a `storage` kötettel együtt a mentés részei, és az általános megőrzés végén törlődnek. Egy futás legfeljebb 120 napot archivál, így egy régi telepítés néhány éjjel alatt éri utol magát; a kézi futtatás `docker compose exec web php artisan hdid:prune`. A Rendszerállapot *Auditnapló* kártyája a tábla méretét, az archívumfájlok számát és méretét és az utolsó archiválást mutatja, és sárgul, ha a kereten túli bejegyzések három napnál tovább az adatbázisban maradnak.

Az auditnapló csendesebb is lett: a gépi kulcs elfogadott használata nem ír bejegyzést (az *API kulcsok* lista „Utoljára használva” oszlopa mutatja, az elutasított és a régi kulcsos használat továbbra is naplózott), az ügyféltörzs-átvétel tételenkénti változásai pedig nem az auditnaplóba, hanem az *Ügyféltörzs-változások* táblába kerülnek (futásonként és tételenként egy sor a tétel, a fiók és a telefonszámok mezőnkénti változásával; kereshető, szűrhető futásra, CSV-be exportálható); az auditnaplóban a futásról egy összegző bejegyzés (`sync.completed`) és a fiókok lezárása, újranyitása, e-mail-változása marad. Az auditnapló oldal alján az utolsó hét nap eseménynevenkénti darabszáma látszik: ha egy név uralja a listát, ott lehet tovább csendesíteni.

A napi 03:30-as tisztítás véglegesen töröl; az eredmény `retention.pruned` eseményként az auditnaplóban áll. A titkot hordozó üzenetek törzse már a kiküldés, a végleges hiba vagy a visszavonás után törlődik. A mentések megőrzését a mentési rend külön szabályozza: az alkalmazás nem törli őket.

---

12 Parancsreferencia

Az alkalmazásparancsok (`hdid:*`) a `docker compose exec web php artisan` előtaggal futnak. A teljes lista a leírásokkal: `docker compose exec web php artisan list hdid` ; egy parancs argumentumai és kapcsolói: `docker compose exec web php artisan help hdid:mfa-reset` . A fiókot módosító parancsok (`hdid:make-admin` , `hdid:mfa-reset` , `hdid:sso-unbind`) minden futása külön auditbejegyzést kap (operációs rendszerbeli felhasználóval és géppel), és e-mailt kap róla minden másik főrendszergazda.

PARANCS	MIT TESZ
`./backup.sh` / `list` / `restore [név]` / `signout`	Mentés a `backup` mappába / a mentések listája / visszaállítás, utána mindenki kiléptetése / kiléptetés visszaállítás nélkül, rövid leállással (7. fejezet).
`docker compose run --rm --no-deps web key`	Új `APP_KEY` és egy `REDIS_PASSWORD` kiírása, csak új telepítéshez.
`docker compose exec web openhdid tls-check` / `reload` / `version`	Tanúsítvány ellenőrzése / nginx újratöltése ellenőrzéssel / verzió.
`hdid:sign-out-all [--force] [--reason=]`	Mindenki kiléptetése mindenhonnan (a mobilalkalmazásban is): munkamenet-korszakok, „emlékezz rám” és mobil tokenek, élő belépési hivatkozások és kódok, a munkamenet-tár és a gyorsítótár (Valkey-nél `FLUSHDB`). Ezt hívja a `./backup.sh restore` és `signout` ; kézzel csak leállított `worker` , `sync-worker` és `scheduler` mellett (`docker compose run --rm --no-deps web artisan hdid:sign-out-all`), mert a gyorsítótár ürítése a futó munka zárjait eldobja. Megerősítést kér (`--force` nélkül), a végén kiírja, mi történt; naplózott (`access.revoked_all`).

PARANCS	MIT TESZ
`hdid:mfa-reset [email] [--off]`	Egy munkatárs második faktorainak törlése, illetve a kétlépcsős munkatársi belépés kikapcsolása vészhelyzetben (például ha csak SMS van engedélyezve, és az átjáró nem működik).
`hdid:sso-unbind [email] [--principal=user|client] [--provider=entra|adfs] [--all] [--force]`	A céges belépés címtár-azonosító kötésének törlése egy fióknál, vagy `--all`-lal egy szolgáltató minden kötésénél (megerősítést kér, `--force` nélkül); a következő belépés újra köt. Minden futás naplózott, a többi főrendszergazda e-mailt kap.
`hdid:make-admin email [--name=] [--force]`	Főrendszergazda létrehozása vagy előléptetése (jelszót kér); a főrendszergazda szerepkört is helyreállítja (újra létrehozza, ha hiányzik, a teljes jogkészletet visszaadja). Pontos címegyezéssel dolgozik (hasonló alakú meglévő fióknál megáll), élesben `.test` címet nem fogad el. Bekapcsolt kétlépcsős belépésnél meglévő fiókot csak `--force`-szal módosít; minden futás naplózott, és a többi főrendszergazda e-mailt kap.
`hdid:health [--strict] [--json]`	Rendszerállapot; kilépési kód 0/1/2.
`hdid:emd-sync [--file=] [--dry-run]`	Ügyféltörzs-átvétel (a régi `hdid:sync` név álnévként működik).
`hdid:emd-sync-token`	Az ügyféltörzs API-tokenjének frissítése, szükség esetén új belépés.
`hdid:test-mail cím [--now]`, `hdid:test-sms szám [--now]`	Próbaüzenet; `--now` a sor megkerülésével.
`hdid:api-key:create "Név" callcenter|mobile_backend`	Gépi kulcs; egyszer jelenik meg, a `mobile_backend` hatókörnél az aláíró titokkal együtt (az aláírás ott kötelező).
`hdid:messages:flush`	Várakozó üzenetek azonnali küldése.
`hdid:prune`	Adatmegőrzési tisztítás és az auditnapló archiválása kézzel (futásonként legfeljebb 120 nap az archívumba).
`schedule:list` (a `scheduler` konténerben)	Ütemezett feladatok és következő futásuk.
`queue:failed` / `queue:retry <uuid>|all` / `queue:forget <uuid>` / `queue:flush` / `queue:prune-failed --hours=48`	Sikertelen feladatok listája, újrafuttatása, törlése egyesével, mind, illetve a régiek törlése. Ugyanez a felületen: *Rendszer › Üzenetsorok* › Sikertelen feladatok; a 30 napnál régebbieket a napi tisztítás magától törli.
`queue:monitor redis:priority,redis:default,redis:sync` (adatbázisos sornál `database:…`) / `queue:clear database --queue=sync --force`	A sorok hossza, illetve egy sor ürítése. Ugyanez a felületen: *Rendszer › Üzenetsorok*, soronként életjellel, darabszámokkal és `Sor ürítése` gombbal; a várakozó feladatok egyesével is törölhetők. A `queue:clear` élesben csak `--force`-szal fut, `exec -T` mellett is.
`down --retry=30`, `up`	Karbantartási mód be és ki (minden példányra).

Rövidítésjegyzék

RÖVIDÍTÉS	JELENTÉS
EMD	Enterprise Master Data; magyarul ügyféltörzs.
AOF	Append Only File – a Valkey tartós naplója.
FPM	FastCGI Process Manager – a PHP folyamatkezelője.
HPA	Horizontal Pod Autoscaler – a Kubernetes automatikus vízszintes skálázója.
LTS	Long-Term Support – hosszú támogatású kiadási ág.
PVC	PersistentVolumeClaim – tartós kötet igénylése Kubernetesben.
RPO / RTO	Recovery Point Objective / Recovery Time Objective – megengedett adatvesztés és helyreállítási idő.
SBOM	Software Bill of Materials – az image összetevőinek jegyzéke.
SemVer	Semantic Versioning – szemantikus verziózás (fő.al.javító).